

Coding Test (CTO Edition):

Event-Driven Microservices with OAuth2/OIDC, SQL/NoSQL & Clean Architecture — AI-Assisted Build

Role Target: Software Development Team Lead

Expected development timeline: 72 hours

Primary Stack (recommendation, but flexible): .NET 8 (C#) or NodeJS + Postgres + MongoDB + Kafka + Keycloak.

AI Requirement: At least **70%** of code must be generated using AI tools (ChatGPT/Copilot/Cursor/etc.). Keep a **Prompt Log** (see Deliverables) and mark AI-generated sections in PR descriptions/commits.

Scenario — “ShopLite”

You’re delivering a minimal **Order & Inventory** domain for a retailer. The platform needs to:

- Accept customer orders. - Reserve stock asynchronously. - Confirm or cancel orders based on availability. - Enforce OAuth2/OIDC with role-based access (customer, admin). - Demonstrate **Clean Architecture** across services.

You will implement **2–3 microservices** and an **event flow** using a broker. Favor **choreography** over orchestration; document trade-offs if you choose otherwise.

Required Services & Responsibilities

1) API Gateway (Edge/BFF)

- Validates JWT via OIDC discovery or static JWKS.
- Routes external requests to downstream services.
- Enforces scopes/roles at the edge. Example scopes: `orders.read`, `orders.write`, `inventory.write`.

2) Orders Service (SQL, Clean Architecture)

- Creates orders and publishes `OrderPlaced` via **outbox pattern**.
- Consumes inventory outcomes to set order status: `CONFIRMED` or `CANCELLED`.
- **SQL model (Postgres recommended):**
 - `orders(id PK, customer_id, status, created_at, updated_at, version)`

- order_items(order_id FK, line_no, sku, qty)
 - outbox(id PK, aggregate_id, type, payload, created_at, processed_at NULL)
- Expose GET /orders/{id} (customer sees own; admin sees any).

3) Inventory Service (NoSQL, Clean Architecture)

- Maintains products collection with available, reserved fields.
- Consumes OrderPlaced → attempts reservation → publishes StockReserved or StockRejected.
- **NoSQL model (MongoDB recommended):**

```
{
  "_id": "ABC",
  "sku": "ABC",
  "available": 10,
  "reserved": 0,
  "updatedAt": "ISO-8601"
}
```

- Admin endpoint to adjust stock: POST /inventory/adjust { sku, delta }.

Optional 4th component: A lightweight **Query/Read Model** projection (SQL or NoSQL) for admin dashboards.

Event Contracts (JSON; versioned)

- OrderPlaced v1

```
{"type":"OrderPlaced","version":1,"orderId":"...","customerId":"...","items":
[{"sku":"ABC","qty":2}], "traceId":"...", "ts":"ISO-8601"}
```

- StockReserved v1

```
{"type":"StockReserved","version":1,"orderId":"...","reservations":[{"sku":"A
BC","qty":2}], "traceId":"...", "ts":"ISO-8601"}
```

- StockRejected v1

```
{"type":"StockRejected","version":1,"orderId":"...","reason":"INSUFFICIENT_ST
OCK","details":[{"sku":"ABC","requested":2,"available":1}], "traceId":"...", "t
s":"ISO-8601"}
```

- OrderConfirmed v1 / OrderCancelled v1 (mirror structure; include final status C reason).

Headers/metadata: include traceId, correlationId and **idempotency key** semantics in handlers.

Functional Endpoints (suggested)

Gateway (public) - POST /orders (role: customer) → 201 { orderId, status: "PENDING"
} - GET /orders/{id} (role: customer own; admin any)

Inventory (admin) - POST /inventory/adjust (role: admin) { sku, delta } - GET
/inventory/{sku} (role: admin)

Health/ops - GET /_health per service (include DB/broker checks). Decide if protected;
document.

Clean Architecture Requirements

- **Layers per service:**
 - **Domain** (entities, value objects, domain events, interfaces)
 - **Application** (use-cases, ports, DTOs, validators)
 - **Infrastructure** (DB/Broker adapters, OIDC/JWT, outbox dispatcher)
 - **Presentation** (HTTP controllers, middlewares, contracts)
- Follow dependency rule: outer depends inward; domain has **no** framework deps.
- Provide a **dependency diagram** and explain any deviations.

Reference folder shape (Orders Service):

```
/orders-service
  /src
    /Domain
    /Application
    /Infrastructure
    /Presentation
  /tests
    /Domain
    /Application
    /Integration
```

OAuth2/OIDC Requirements

- Implement JWT validation using **OIDC discovery** (Keycloak) or static JWKS for local.
 - Validate iss, aud, exp, and roles/scopes. Cache JWKS; document rotation strategy.
 - Provide **sample tokens** or a script to mint tokens for customer and admin.
-

Messaging G Reliability

- Use Kafka/RabbitMQ/Redis Streams **or** a DB Outbox + background dispatcher.
 - At-least-once delivery; implement **idempotent** consumers.
 - Dead-letter strategy (log + retry/backoff; DLQ if broker supports it).
-

Observability

- Correlated, structured logs (include `traceId`, `orderId`).
 - Basic metrics (request count, error rate, reservation success %).
 - Health checks (DB, broker). If using .NET: `AspNetCore.Diagnostics.HealthChecks` is fine.
-

Deliverables

- **Code repo** with **README.md** including:
 - Architecture overview, trade-offs, and **Clean Architecture** explanation.
 - How to run: `docker compose up` that launches DBs, broker, OIDC (Keycloak container or mock), services.
 - How to test:
 - `cURL`/REST Client or Postman collection for acceptance scenarios.
 - Unit + integration test commands.
 - Known limitations C next steps.
 - **Diagrams**: C4 Context + Container, plus a **sequence diagram** for the saga.
 - **Prompt Log (mandatory)** — a markdown file with:
 - Each **AI prompt** you used and the **tool name**.
 - The **result summary** and **manual edits** (if any).
 - Why the prompt was needed (architecture/code/test/docs).
 - **AI Usage Declaration** in the README header with approximate percentage of AI-generated code.
-

Evaluation Rubric (100 pts)

Area	Points	What we look for
Architecture G Clean Architecture	25	Clear boundaries; dependency rule respected; adapters/ports well defined; diagrams.
Event-Driven Design G	20	Outbox/idempotency;

Area	Points	What we look for
Reliability		well-versioned events; clear choreography; retry/DLQ.
OAuth2/OIDC Security	15	Correct JWT validation; scopes/roles; sample tokens; secure defaults.
Database Design (SQL & NoSQL)	15	Normalized SQL model; suitable NoSQL schema; indexes & migrations.
Code Quality & Tests	15	Unit & integration tests; event contract tests; clean, readable code.
Operations (Docker/Health/Logs/Metrics)	10	One-command up; health checks; structured logging; metrics present.

Bonus (+30 pts max) — Ability to make well-reasoned assumptions when the test is ambiguous. Bonus credit for documenting rationale clearly.

Anti-Cheating & AI-Usage Policy

- Submit **Prompt Log** and keep **commits small** with descriptive messages.
 - Label AI-generated commits with AI: prefix (example: AI: generate OrdersController and tests).
 - We will conduct a **30-minute follow-up** to dive into: why specific patterns, how you'd evolve schemas, failure modes.
 - We may ask you to **extend** a function live to confirm authorship.
-

Starter Prompts (optional, to speed you up)

- *Architecture*: "Generate a Clean Architecture folder structure for a .NET 8 microservice named Orders with Domain, Application, Infrastructure, Presentation; include DI setup and example entity."
- *Outbox*: "In .NET 8 with EF Core and Postgres, implement an Outbox pattern with a background dispatcher and exactly-once-ish semantics; include migration and tests."
- *Kafka Consumer*: "Create a resilient Kafka consumer in .NET 8 using Confluent.Kafka with retry/backoff, idempotency key, and DLQ topic."

- *JWT Validation*: “Validate JWT from Keycloak discovery endpoint in ASP.NET Core; enforce scope orders.write on POST /orders.”
 - *Mongo Repository*: “MongoDB repository for Product with optimistic concurrency and indexes on sku and available.”
 - *Sequence Diagram*: “PlantUML sequence diagram for OrderPlaced → StockReserved/StockRejected → OrderConfirmed/Cancelled.”
-

Repository Layout (example)

```
/gateway
/orders-service
/inventory-service
/contracts
/ops/docker-compose.yml
/tests
/docs
README.md
PROMPTS.md # Prompt Log
```

Grader’s Checklist (copy into PR template)

- ☐ docker compose up boots broker, DBs, OIDC, services
 - ☐ Tokens minted and roles enforced
 - ☐ Happy path passes via collection/script
 - ☐ Insufficient stock scenario passes
 - ☐ Idempotency verified
 - ☐ Clean Architecture boundaries clear
 - ☐ Outbox/consumer idempotency
 - ☐ Structured logs + health checks
 - ☐ Diagrams present
 - ☐ Prompt Log present; AI usage $\geq 70\%$
-

Submission

- Share a Git repo
- Include README.md, PROMPTS.md, diagrams, and a short Loom/Walkthrough (≤ 5 min) **optional**.